

Linguagem C e sistemas embarcados



Conversão de bases



Bases numéricas

Decimal	Hexadecimal	Binário
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

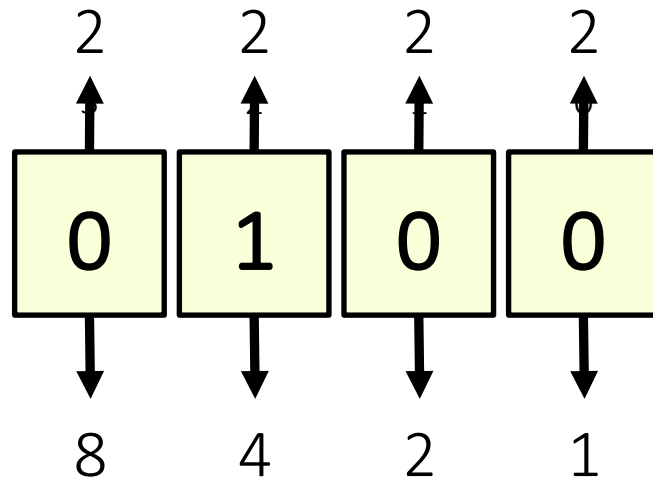
Sistema octal

Decimal	Octal	Binário
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111



Conversão

Convertendo binário para hexadecimal



1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Conversão

Binário ➡ Hexadecimal

Binário

0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---

4

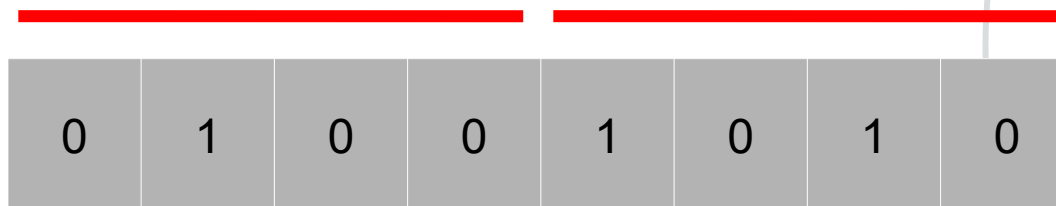
A

Conversão

Hexadecimal ➡ Binário

Hexadecimal

4 A



0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---

Valores na linguagem C padrão

Decimal

```
int x = 10;
```

Hexadecimal

```
int x = 0x0A;
```

Octal

```
int x = 012;
```


Variáveis

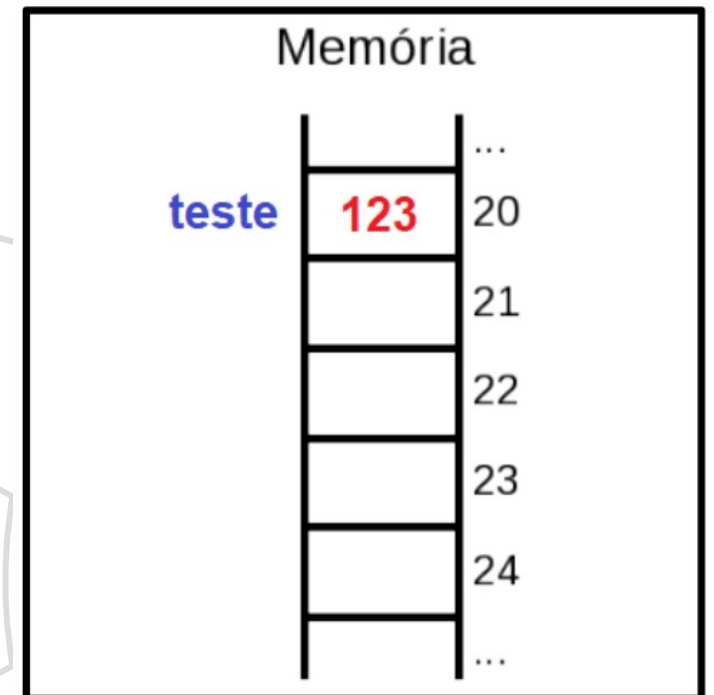


Variáveis

Espaço na memória com um valor que pode ser alterado

- Uma variável em linguagem C possui:
 - Um tipo que indica o tamanho e formato.
 - Um nome para referenciar o conteúdo.
 - Um espaço na memória para armazenar seu valor.

Exemplo: char teste = 123;



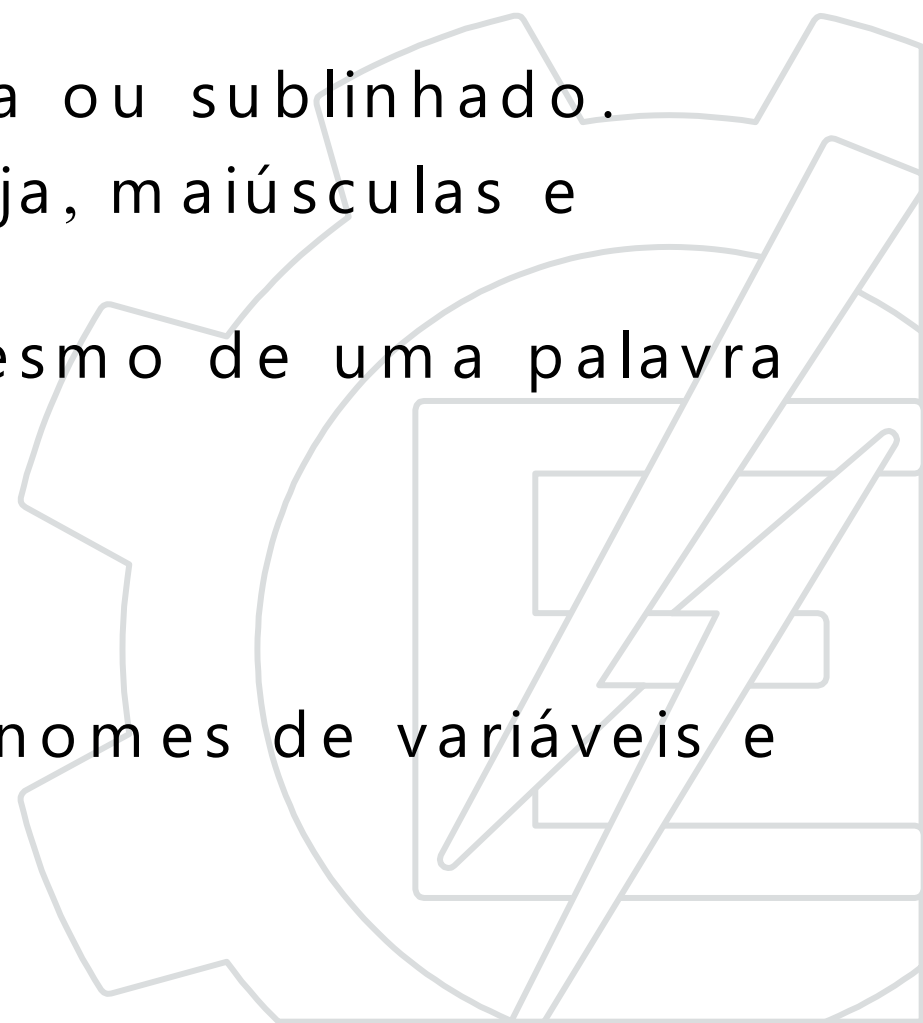
Nomes das Variáveis

Regras:

- O número de caracteres não importa.
- O primeiro caractere deve ser uma letra ou sublinhado.
- A linguagem C é “case sensitive”, ou seja, maiúsculas e minúsculas são diferentes.
- O nome da variável não pode ser o mesmo de uma palavra reservada em linguagem C.

Boa prática:

- Utilize principalmente minúsculas para nomes de variáveis e apenas MAIÚSCULAS para constantes.



Variáveis

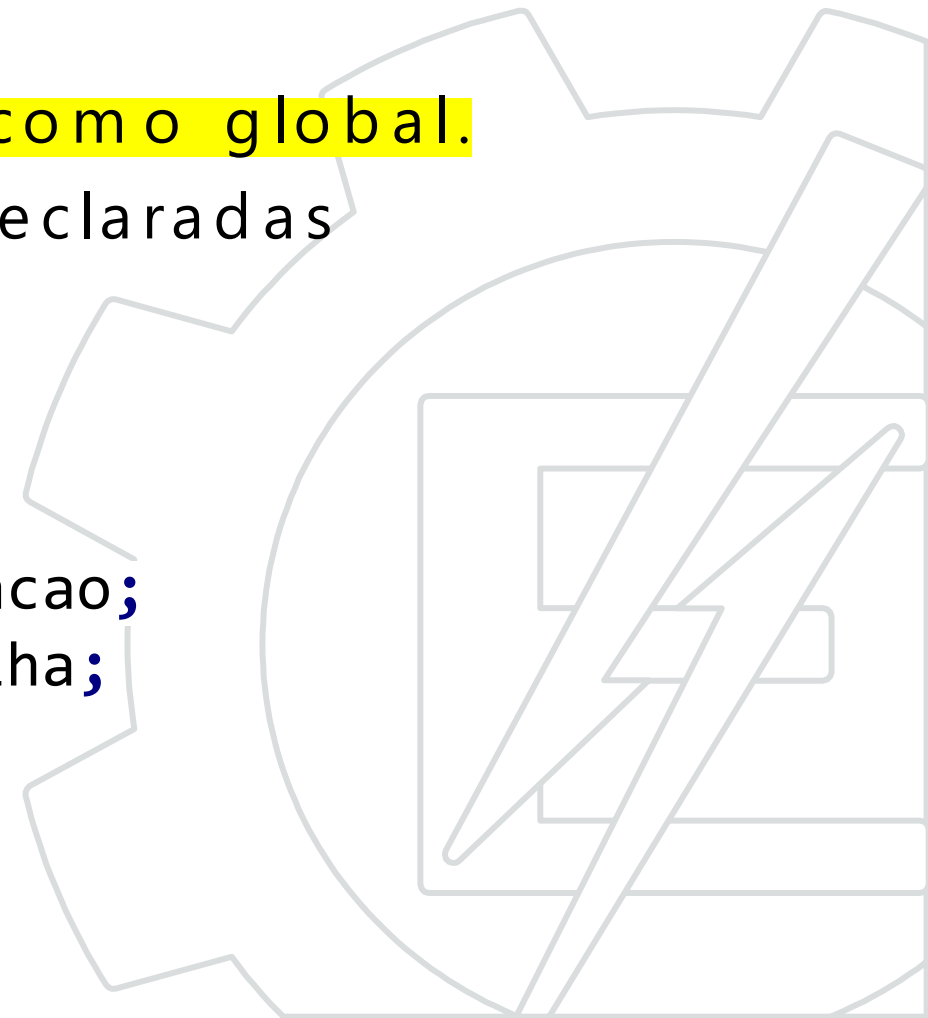
- Criadas da seguinte forma:

tipo nome;

Declarar no início da função ou como global.

- Variáveis do mesmo tipo podem ser declaradas separadas por vírgula.

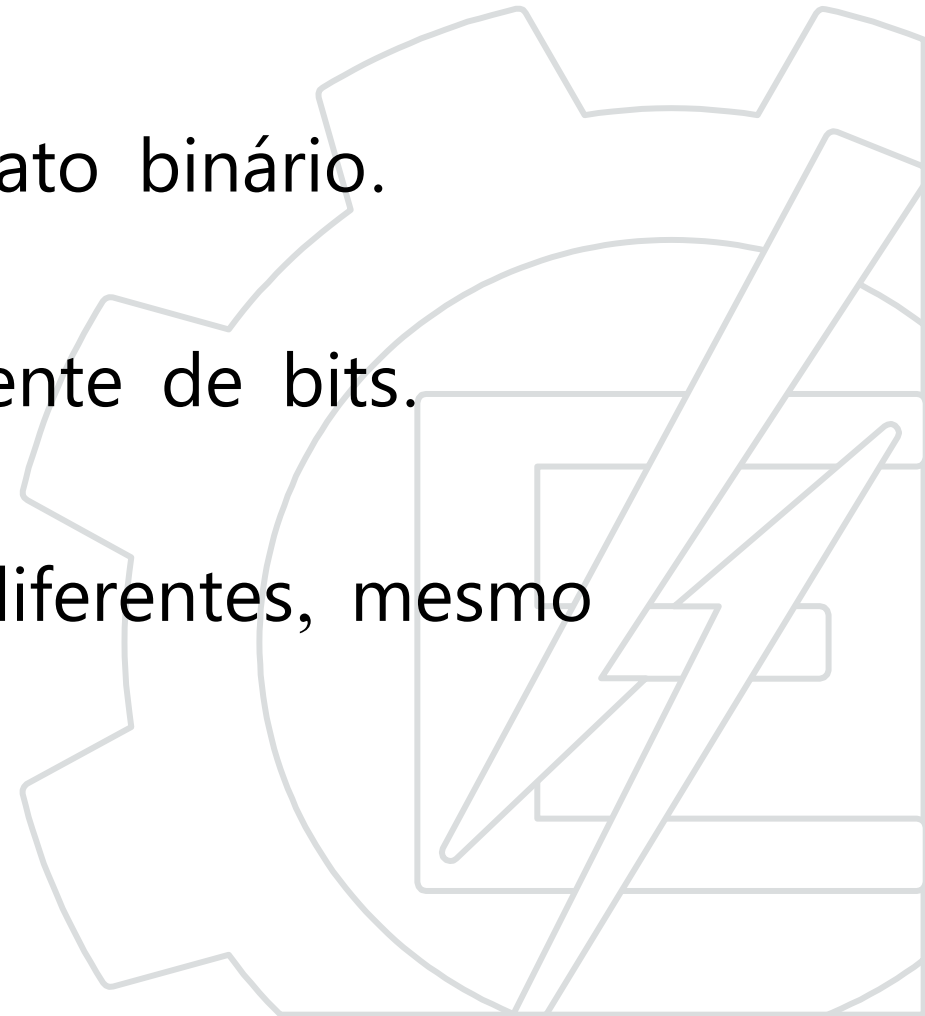
```
void main(void)  
{  
  int numFuncionarios;  
  float salarioMinimo, bonificacao;  
  double imposto, descontoEmFolha;  
}
```



Variáveis

Observações importantes

- Toda informação é armazenada em formato binário.
- Cada tipo contém uma quantidade diferente de bits.
- Cada bit pode representar informações diferentes, mesmo dentro de uma mesma variável.



Tipos básicos da linguagem C

Tipo	Bits	Bytes	Faixa de valores
char	8	1	-128 à 127
int	16	2	-32.768 à 32.767
float	32	4	-3.4×10^{-38} à 3.4×10^{38} ;
double	64	8	-3.4×10^{-308} à 3.4×10^{308}

Tipos básicos da linguagem C



Imagem: www.graphicsfactory.com

Tipo	Bits	Bytes	Faixa de valores
char	8	1	-128 à 127
int	16	2	-32.768 à 32.767
float	32	4	-3.4×10^{-38} à 3.4×10^{38} ;
double	64	8	-3.4×10^{-308} à 3.4×10^{308}

Cuidado para não ultrapassar

Atribuição de valores

- Inicializar uma variável é atribuir um valor a esta no momento de sua declaração.

```
void main(void)
{
    int numFuncionarios = 2;
    float salarioMinimo = 510.0;
    double imposto = 0.25;
    double descontoEmFolha = 151.97;
}
```



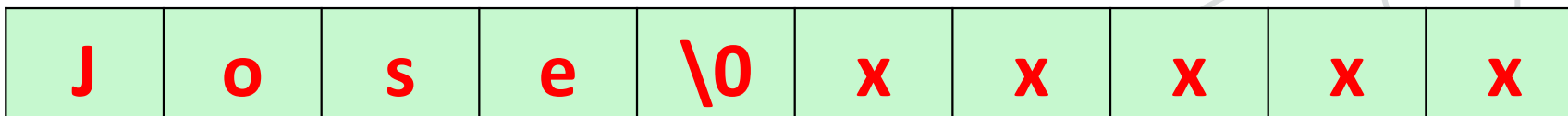
Atribuição de valores

- Para armazenar textos utilizamos um vetor de caracteres.
- A inicialização pode ser feita utilizando o texto entre **aspas duplas**.

```
void main(void)
{
    char nome[10] = "Jose";
}
```

Na memória a palavra Jose ocupa 5 bytes.

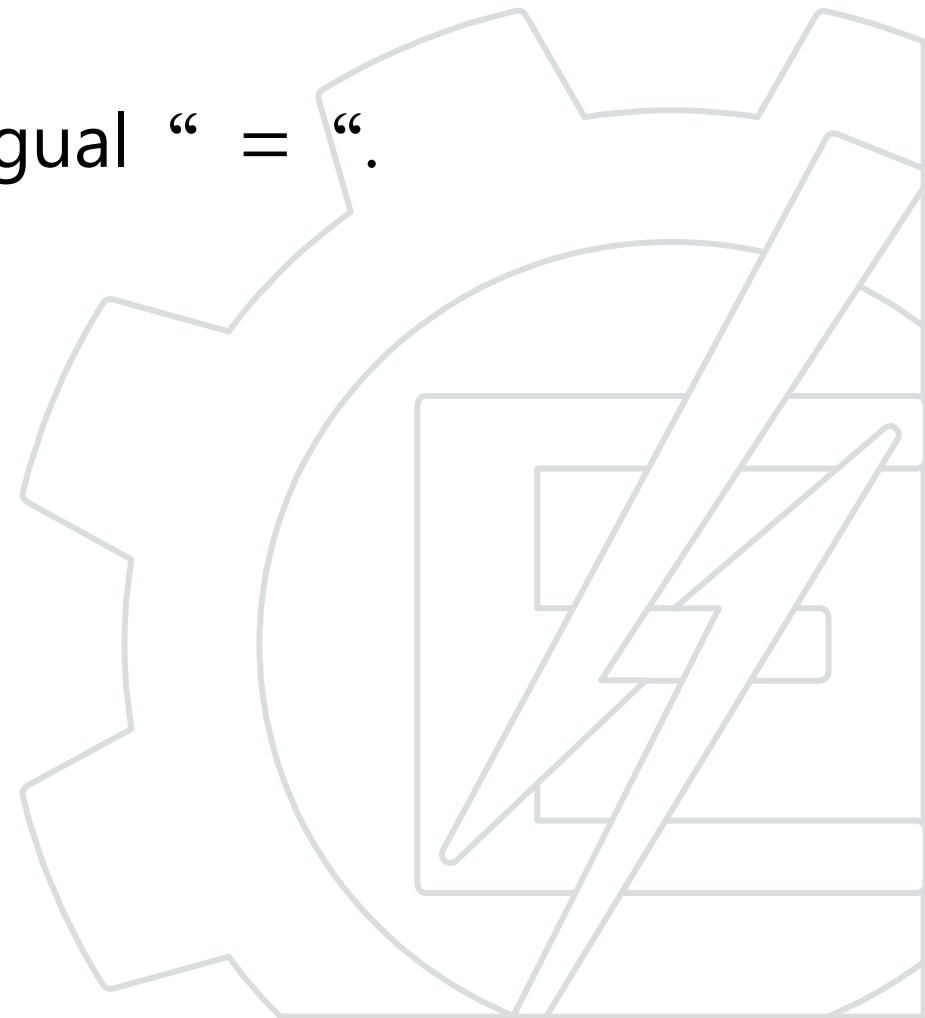
nome



Atribuição de valores

- A atribuição é uma alteração do valor da variável no meio do programa.
- O operador de atribuição é o sinal de igual “ = ”.

```
void main(void)
{
    float valor;
    valor = 1234.56;
    valor = valor * 1.25;
    ...
}
```

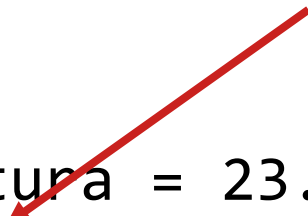


Alteração de tipo

- O typecast força a expressão a ser de um determinado tipo.
- Basta colocar o tipo desejado entre parênteses antes do valor.

```
void main(void)
{
    int ajuste;
    float temperatura = 23.94;
    ajuste = (int) temperatura * 45;
    // Resultado ajuste = 1035
    // a conversão faz perder precisão
    ...
}
```

Typecast



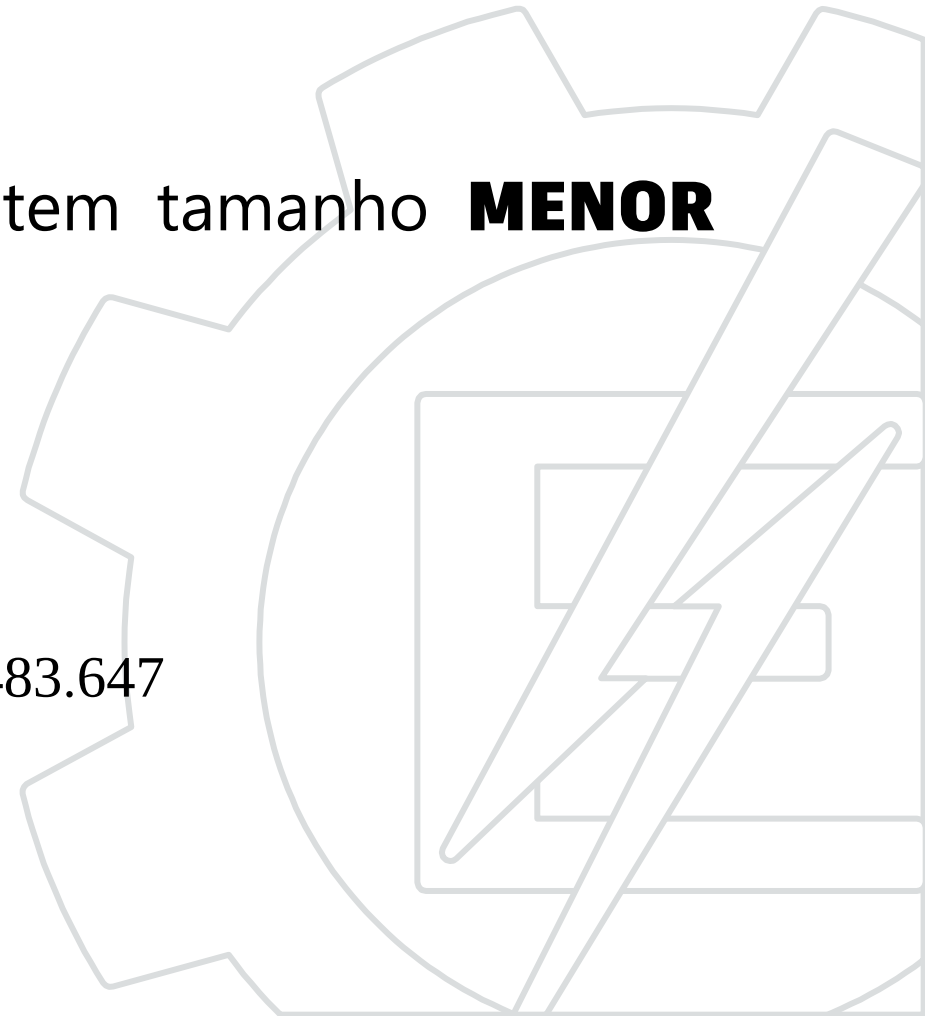
Modificadores de tamanho

long: Uma variável com modificador “long” tem tamanho **MAIOR** ou **IGUAL** ao tipo básico da variável.

short: Uma variável com modificador “short” tem tamanho **MENOR** ou **IGUAL** ao tipo básico da variável.

Exemplo:

- short int -> 2 bytes: de -32.768 à 32.767
- int -> 2 bytes: de -32.768 à 32.767
- long int -> 4 bytes: de -2.147.483.648 à 2.147.483.647



Modificadores de sinal

signed: Pouco usado. Já é definido por default.

unsigned: Indica que uma variável não suporta valor negativo, usado com “int” e “char”.

Tipo	Tamanho	Valores
char	1 byte	-127 a 127
unsigned char	1 byte	0 a 255
int	2 bytes	-32.767 a 32.767
unsigned int	2 bytes	0 a 65.535

Exemplo de uso dos modificadores

Tipo	Bytes	Excursão máxima
unsigned char	1	0 a 255
signed char	1	-128 a 127
unsigned int	2	0 a 65.535
signed int	2	-32.768 a 32.767
long int	4	-2.147.483.648 a 2.147.483.647
unsigned long int	4	0 a 4.294.967.295
short int	2	-32.768 a 32.767



Tamanho fixo?



Tamanhos “fixos”

Categoria	Com sinal			Sem sinal		
	Tipo	Valor mínimo	Valor máximo	Tipo	V. mínimo	Valor máximo
Tamanho exato	<code>intn_t</code>	<code>INTn_MIN</code>	<code>INTn_MAX</code>	<code>uintn_t</code>	0	<code>UINTn_MAX</code>
No mínimo X	<code>int_leastn_t</code>	<code>INT_LEASTn_MIN</code>	<code>INT_LEASTn_MAX</code>	<code>uint_leastn_t</code>	0	<code>UINT_LEASTn_MAX</code>
Mais rápido (mínimo X)	<code>int_fastn_t</code>	<code>INT_FASTn_MIN</code>	<code>INT_FASTn_MAX</code>	<code>uint_fastn_t</code>	0	<code>UINT_FASTn_MAX</code>
Ponteiro	<code>intptr_t</code>	<code>INTPTR_MIN</code>	<code>INTPTR_MAX</code>	<code>uintptr_t</code>	0	<code>UINTPTR_MAX</code>
Maior tamanho	<code>intmax_t</code>	<code>INTMAX_MIN</code>	<code>INTMAX_MAX</code>	<code>uintmax_t</code>	0	<code>UINTMAX_MAX</code>

- A partir do padrão C99 foram criados novos **tipos de inteiros** com tamanho fixo.

- Estes tipos são definidos no header `<stdint.h>`

<stdint.h>

```
#include <features.h>
#include <bits/wchar.h>
#include <bits/wordsize.h>

/* Exact integral types.  */

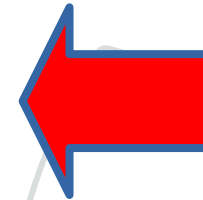
/* Unsigned.  */
typedef unsigned char          uint8_t;
typedef unsigned short int     uint16_t;
#ifndef __uint32_t_defined
    typedef unsigned int       uint32_t;
    #define __uint32_t_defined
#endif
#if __WORDSIZE == 64
    typedef unsigned long int   uint64_t;
#else
    __extension__
    typedef unsigned long long int uint64_t;
#endif
```

Signed

int8_t
int16_t
int32_t
int64_t

Unsigned

uint8_t
uint16_t
uint32_t
uint64_t



Modificadores de Acesso

- Os modificadores de acesso modificam o comportamento de leitura/escrita da variável.

Exemplo:

```
void main(void)
{
    unsigned char x;
    while (x!=x);
}
```



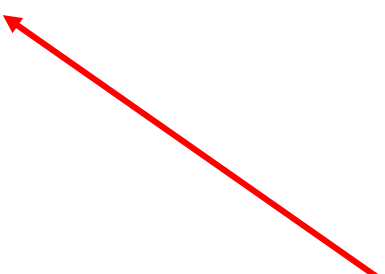
O que isso gera?



Modificadores de Acesso

- Compilando o código anterior temos em assembly:

```
// Starting pCode block  
S_Testes__main code  
_main:  
.line 19 // while (X!=X);  
RETURN
```



O programa termina
sem fazer nada.

Modificadores de Acesso

- O volatile força a leitura.
- As variáveis associadas as portas devem ser volatile.

```
void main(void)
{
    volatile unsigned char x;
    while (x!=x); ← O que isso gera?
}
```



```
S_Testes__main code // Starting pCode block
_main:
_00105_DS_:
.line 19 // Teste.c com while (X != X);
MOVLW 0x83 //primeira parte do endereço
MOVWF r0x00
MOVLW 0x0f //segunda parte do endereço
MOVWF r0x01
MOVFF r0x00, FSR0L
MOVFF r0x01, FSR0H
MOVFF INDF0, r0x00 //realiza primeira leitura
MOVLW 0x83 //primeira parte do endereço
MOVWF r0x01
MOVLW 0x0f //segunda parte do endereço
MOVWF r0x02
MOVFF r0x01, FSR0L
MOVFF r0x02, FSR0H
MOVFF INDF0, r0x01 //realiza segunda leitura
MOVF r0x00, W
XORWF r0x01, W
BNZ _00105_DS_ //faz o teste para igualdade
RETURN
```

Modificadores de Acesso



Modificadores de acesso

O modificador `const` evita a mudança de um valor constante.

- Exemplo de uso do modificador `const`:

```
void main(void)
{
    const unsigned char x = 5;
    x = 3;
}
```



A compilação resulta em erro, já que a variável 'x' não pode ser alterada.



Modificadores de acesso

O `const` é interessante para evitar que um ponteiro (endereço da variável) seja modificado quando é passado para uma função.

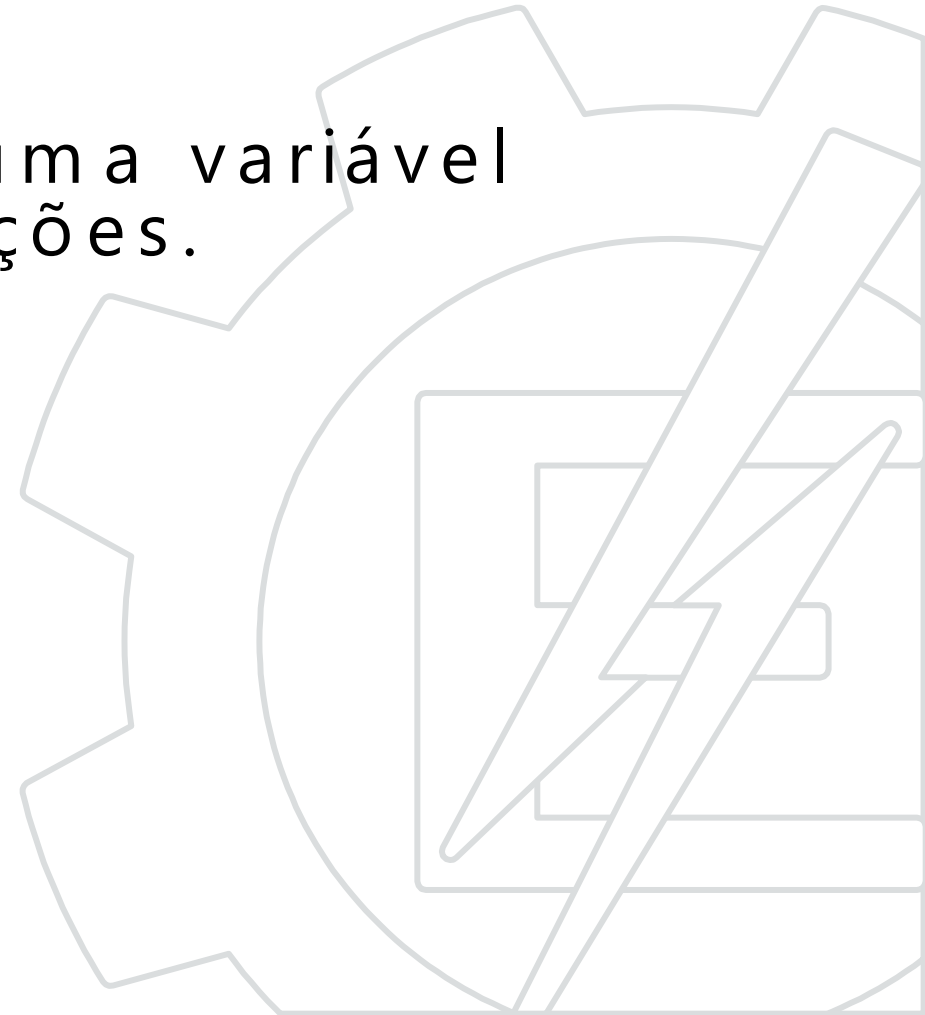
```
int soma(const int *a, const int *b)
{
    return *a + *b;
}
```

- Neste exemplo o valor do endereço não pode ser modificado dentro da função.

Modificador de persistência

- Uma variável do tipo static mantém o valor da variável mesmo depois que a função termina.
- Ao contrário da variável global uma variável static não é vista por outras funções.

```
void  
mostra_valor(void)  
{  
    static int x=0;  
    printf("%d",x);  
    x++;  
}
```



Operação com variáveis

- Quando uma informação é passada para o formato digital (que tem um tamanho finito), pode ocorrer perda de:
 - informação
 - Resolução
 - Amplitude

Exemplo de limitação nos sistemas digitais: Depois de quantas iterações os loops param?

```
float x = 0;  
while (x != 4) {  
    lcdChar('*');  
    x += 0.4f;  
}
```

```
char x = 0;  
while (x < 200) {  
    lcdChar('*');  
    x++;  
}
```

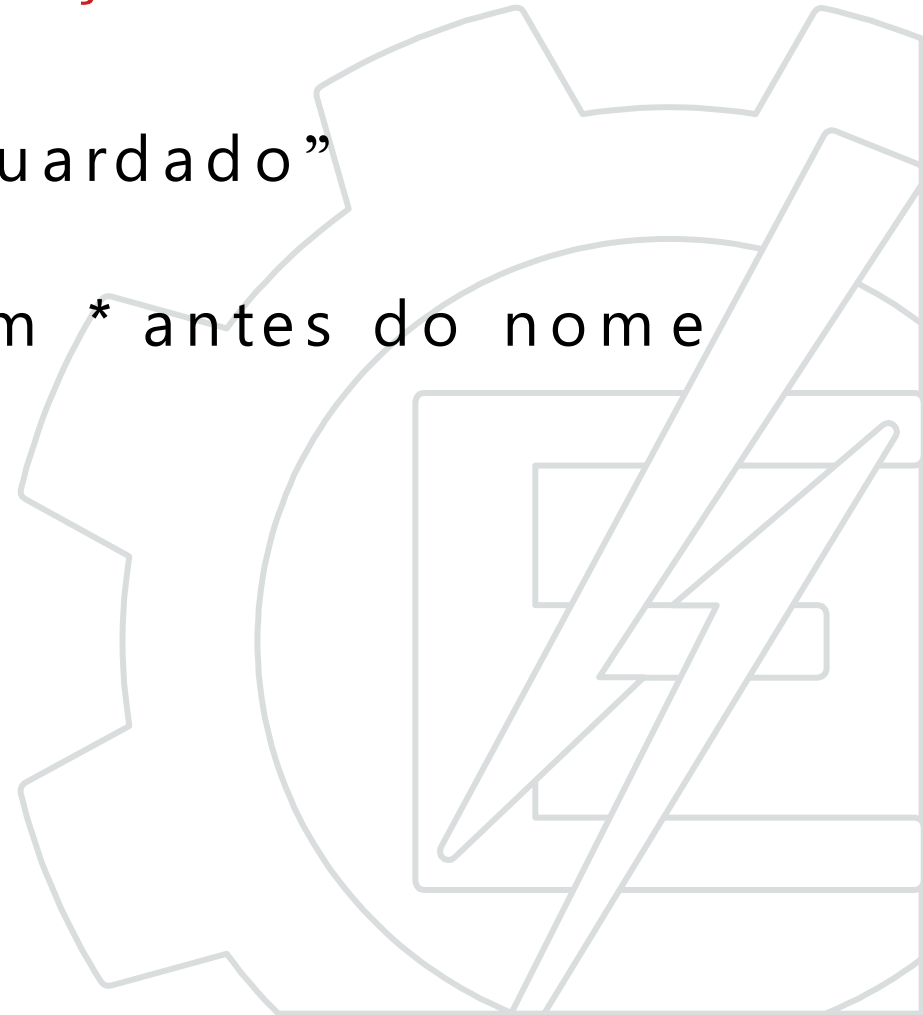
Ponteiros e Referências



Ponteiros

- Variáveis que **guardam um endereço** de memória
 - O tipo especifica o “tipo do valor guardado”
 - os ponteiros são declarados com um `*` antes do nome da variável

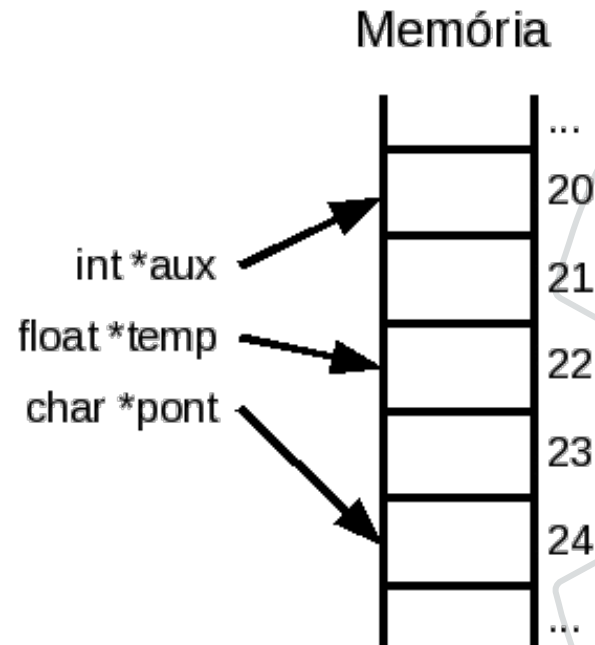
```
tipo  
*nomeVariavel;
```



Ponteiros

- São variáveis que **armazenam endereços de memória e não valores** do tipo char, int, double ou float.
- O * é usado para criar a variável do tipo ponteiro.

```
int *aux;  
float *temp;  
char *pont;
```



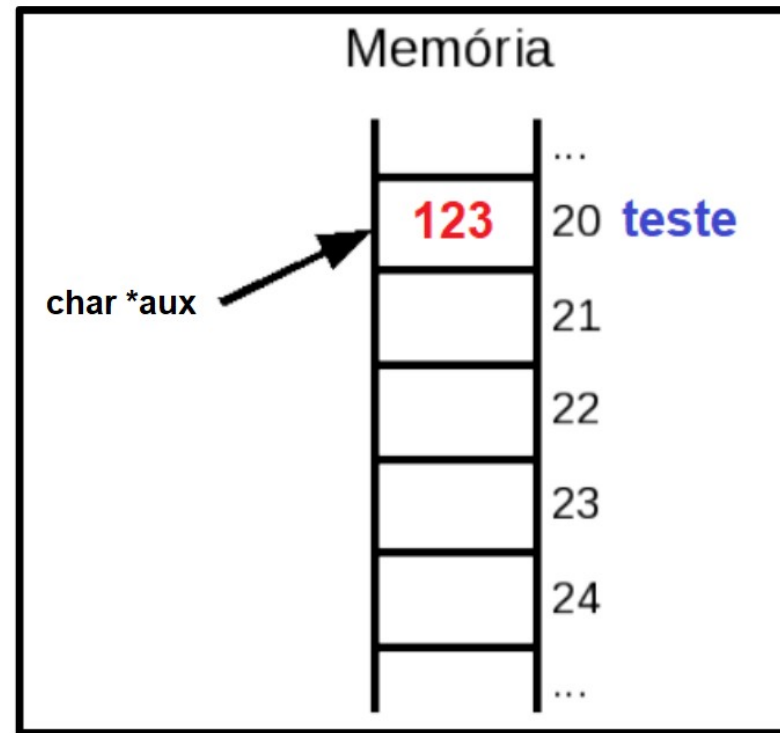
Ponteiros

O * é usado também para referenciar o conteúdo da memória.

```
char *aux, teste;  
aux = &teste;  
*aux = 123;  
printf("%d", *aux);
```



O conteúdo apontado pelo
ponteiro aux

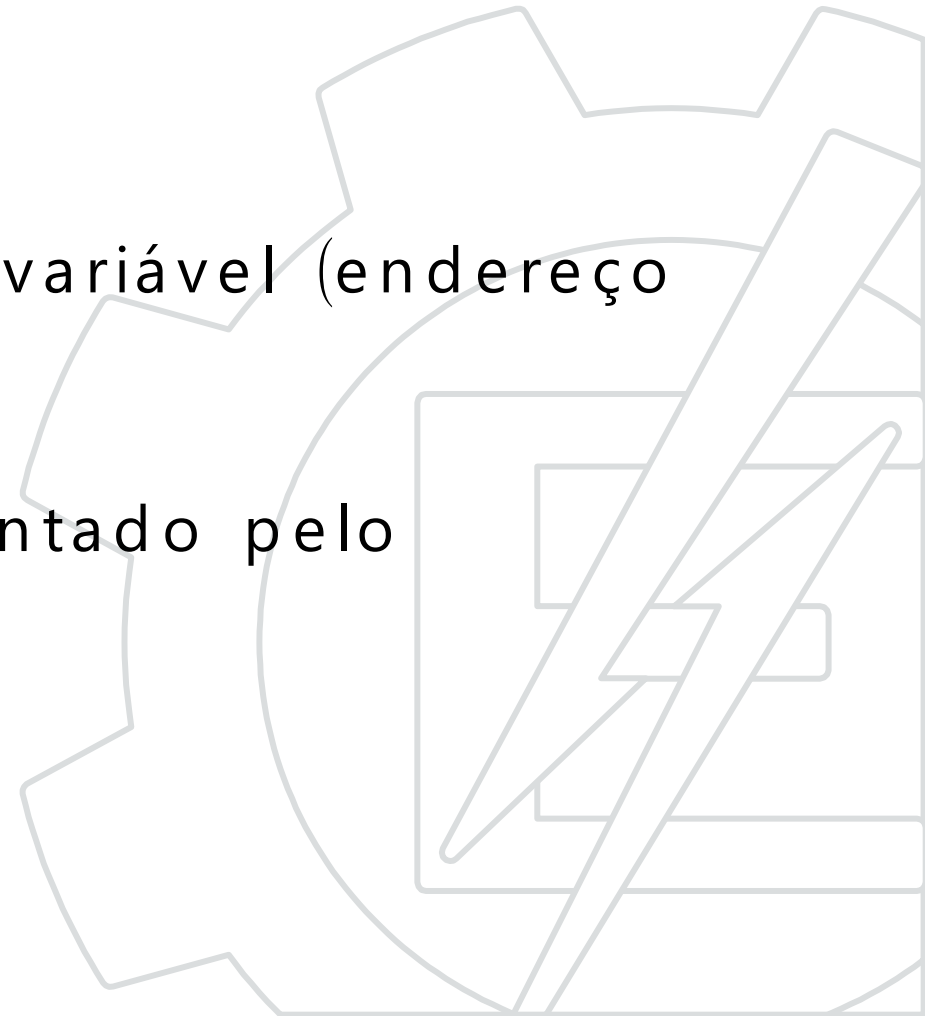


Os operadores e os ponteiros

Resumindo:

Operador `&` → obtém o endereço da variável (endereço de).

Operador `*` → acessa o conteúdo apontado pelo ponteiro (o conteúdo de).



Exemplo de uso dos ponteiros

```
void main(void){  
    int x=10;  
    int *p = &x;  
  
    lcdInit();  
    *p = 20; //ou p1[0] = 20;  
  
    printf("%d é igual a %d",x,*p);  
}
```

O endereço de x



O conteúdo no endereço
apontado por "p"



Até a próxima.

